RAG Production App

Technical Documentation – For IntellifyAI Chatbot Task

Stack	Pycharm-Python 3.13, FastAPI, Streamlit, Inngest, Qdrant, Gemini AI
Author	Binayak Pradhan (For task given by IntellifyAI Team)
Purpose	Upload PDFs, ingest them into a vector store, and answer questions using RAG

1. Project Overview

The RAG Production App is an end-to-end Retrieval-Augmented Generation system that allows users to upload PDF documents, process and index their content in a vector database, and pose natural-language questions that are answered using the most relevant document chunks as context for a large language model.

The system is built around a decoupled event-driven architecture using **Inngest** for durable function orchestration, **Qdrant** as the vector store, **sentence-transformers** for local embedding generation, and the **Gemini API** (via an OpenAI-compatible endpoint) for answer synthesis. A **Streamlit** frontend provides the user interface, while a **FastAPI** application hosts the Inngest function workers.

Key Capabilities

PDF Ingestion	Upload any PDF; it is chunked, embedded locally, and stored in Qdrant automatically.
Semantic Search	Queries are embedded with the same model and the closest document chunks are retrieved.
LLM Answer Synthesis	Retrieved context is forwarded to Gemini to produce a grounded, concise answer.
Observability	Every step runs inside a tracked Inngest function, providing full audit trails and retries.
Scalable Storage	Qdrant supports large document collections with cosine-similarity search.

2. System Architecture

The system separates concerns cleanly into three layers: a presentation layer (Streamlit), an orchestration / worker layer (Inngest + FastAPI), and a persistence layer (Qdrant). Communication between the frontend and workers happens exclusively through Inngest events, making the pipeline observable, retryable, and easy to extend.

Component Interaction

Component	Role	Technology
Frontend UI	Upload PDFs, submit queries, display answers	Streamlit
Event Bus	Route events between UI and workers; retry on failure	Inngest
API Worker	Host Inngest functions; orchestrate steps	FastAPI + Uvicorn

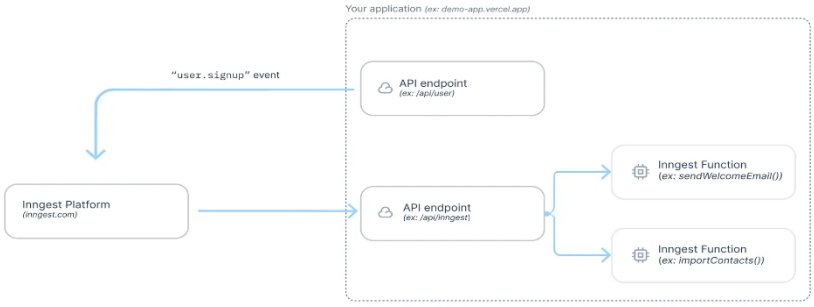
Embedder	Convert text chunks and queries to dense vectors	sentence-transformers (all-MiniLM-L6-v2)
Vector Store	Persist and search document embeddings	Qdrant (local HTTP)
LLM	Generate grounded answers from retrieved context	Gemini via OpenAI-compat endpoint
PDF Loader	Parse and chunk raw PDFs	llama-index + SentenceSplitter

Ingestion Data Flow

1. User selects a PDF in the Streamlit UI.
2. The file is saved to the local *uploads/* directory.
3. A **rag/ingest_pdf** Inngest event is fired with the file path and source ID.
4. The **RAG: Ingest PDF** function picks up the event and executes two steps:
   • **load-and-chunk**: PDFReader parses the file; SentenceSplitter produces 1000-token chunks with 200-token overlap.
   • **embed-and-upsert**: all-MiniLM-L6-v2 embeds every chunk (dim=384); deterministic UUIDs are computed; points are upserted into Qdrant.
5. The result (number of ingested chunks) is returned and visible in the Inngest dashboard.

Query Data Flow

1. User types a question and selects top-k in the Streamlit UI.
2. A **rag/query_pdf_ai** Inngest event is fired.
3. The **RAG: Query PDF** function executes two steps:
   • **embed-and-search**: the question is embedded; Qdrant returns the top-k nearest chunks with their source metadata.
   • **llm-answer**: the chunks are assembled into a context block and sent to Gemini via the Inngest AI adapter.
4. The answer and source list are returned as the run output.
5. The Streamlit UI polls the Inngest local API until the run completes, then renders the answer.



Inngest architecture flow with User and API server

3. Codebase Structure

File	Purpose
app.py	Streamlit frontend — file upload, event dispatch, polling, answer display
main.py	FastAPI app + Inngest functions for ingestion and querying
data_loader.py	PDF loading, sentence splitting, and local embedding (sentence-transformers)
vector_db.py	QdrantStorage wrapper — collection management, upsert, cosine search
custom_types.py	Pydantic models for inter-step data contracts
pyproject.toml	Project metadata and dependency declarations (uv / PEP 517)
.env	Runtime secrets: GEMINI_API_KEY, INNGEST_API_BASE, etc.

4. Module Reference

4.1 data_loader.py

Responsible for all document processing and embedding. The SentenceTransformer model and SentenceSplitter are instantiated once at module load to avoid repeated initialisation overhead.

Function	Signature	Description
load_and_chunk_pdf	load_and_chunk_pdf(path: str) -> list[str]	Reads a PDF via LlamaIndex PDFReader, splits all page texts into overlapping 1000-token chunks (200-token overlap) using SentenceSplitter, and returns a flat list of chunk strings.
embed_texts	embed_texts(texts: list[str]) ->list[list[float]]	Encodes a list of text strings with all-MiniLM-L6-v2 (384-dimensional output) and returns the embeddings as a plain Python list of float lists.

vector_db.py — QdrantStorage

Thin synchronous wrapper around the Qdrant Python client. Automatically creates the target collection if it does not exist, using cosine distance with the configured embedding dimension.

Method	Signature	Description
__init__	_init_(url, collection, dim)	Connects to Qdrant at the given URL, creates the collection with COSINE distance if absent.
upsert	upsert(ids, vectors, payloads)	Inserts or updates a batch of PointStruct objects with associated text and source metadata.
search	search(query_vector, top_k) -> dict	Queries the collection for the top-k nearest neighbours; returns {'contexts': [...], 'sources': [...]}.

4.2 custom_types.py — Pydantic Models

All inter-step data is typed with Pydantic models, enabling Inngest's serialiser to validate and reconstruct objects across function step boundaries.

Model	Fields	Used by
RAGChunkAndSrc	chunks: list[str], source_id: str	load-and-chunk step output
RAGUpsertResult	ingested: int	embed-and-upsert step output
RAGSearchResult	contexts: list[str], sources: list[str]	embed-and-search step output
RAGQueryResult	answer: str, sources: list[str], num_contexts: int	Final query function return

5. Inngest Functions

Inngest provides durable execution: each step is automatically retried on failure, its output is persisted, and the full run history is visible in the Inngest Dev Server dashboard at **http://127.0.0.1:8288**.

5.1 RAG: Ingest PDF

Trigger event: rag/ingest_pdf

Event payload fields:

- pdf_path— absolute path to the saved PDF file
- source_id— identifier stored in Qdrant payload (defaults to filename)

Steps:

Step ID	Action	Output type
load-and-chunk	Calls load_and_chunk_pdf(); returns chunk list with source ID	RAGChunkAndSrc
embed-and-upsert	Embeds chunks, assigns deterministic UUIDs, upserts to Qdrant	RAGUpsertResult

5.2 RAG: Query PDF

Trigger event: rag/query_pdf_ai

Event payload fields:

- question— the user's natural-language question
- top_k— number of chunks to retrieve from Qdrant (default 5)

Steps:

Step ID	Action	Output type
embed-and-search	Embeds question, queries Qdrant for top-k chunks	RAGSearchResult
llm-answer	Sends context + question to Gemini via Inngest AI adapter; extracts answer text	dict (answer, sources)

LLM Configuration

The query function uses Inngest's built-in AI adapter for LLM calls. The adapter is configured to point at Google's OpenAI-compatible endpoint:

```
adapter = ai.openai.Adapter(
    auth_key = os.getenv("GEMINI_API_KEY"),
    base_url = "https://generativelanguage.googleapis.com/v1beta/openai/", model =
    "gemini-3.1-flash-lite-preview",
)
```

The system prompt constrains the model to answer only from provided context. Temperature is set to 0.2 for factual, low-variance responses. Max tokens is capped at 1024.

6. Configuration & Environment

All sensitive values are loaded from a .env file via python-dotenv. The table below lists every recognised environment variable.

Variable	Required	Default	Description
GEMINI_API_KEY	Yes	—	API key for Google Gemini (used in Inngest AI adapter)
INNGEST_API_BASE	No	http://127.0.0.1:8288/v1	Base URL of the Inngest Dev Server REST API (used for polling run output)
QDRANT_URL	No	http://localhost:6333	URL of the running Qdrant instance (hardcoded default in QdrantStorage)

Chunking Parameters

In data_loader.py, the SentenceSplitter is initialised with chunk_size=1000 and chunk_overlap=200. These control how aggressively the PDF text is segmented. Larger chunks preserve more context per retrieval hit; greater overlap reduces the chance of splitting a sentence across two chunks. Adjust these constants to tune the quality vs. cost trade-off for your documents.

Qdrant Collection

The default collection name is "docs" with a vector dimension of 384 (matching all-MiniLM-L6-v2 output). The collection is created automatically on first ingest. If you change the embedding model you must also update dim in QdrantStorage.

7. Setup & Running Locally

Prerequisites

- Python 3.13+
- uv (recommended) or pip for dependency management
- Docker — for running Qdrant
- Inngest Dev Server (npx inngest-cli@latest dev)
- A valid Google Gemini API key or any other API Key (Eg- OpenAI, Groq, Anthropic)

Installation

1. Clone the repository

```
git clone <repo-url> && cd rag-production-app
```

2. Install dependencies

```
uv add  #dependencies or: pip install #dependencies
```

3. Configure environment

```
cp .env.example .env
# Edit .env and set GEMINI_API_KEY
```

4. Start Qdrant

```
docker run -d --name qdrantRagDB -p 6333:6333 -v "$(pwd)/qdrant_storage:/qdrant/storage" qdrant/qdrant
```

5. Start Inngest Dev Server

```
npx inngest-cli@latest dev -u http://127.0.0.1:8000/api/inngest --no-discovery
```

6. Start the FastAPI worker

```
uv run uvicorn main:app
```

7. Start the Streamlit UI

```
uv run streamlit run .\app.py
```

8. Dependencies

Package	Min Version	Purpose
fastapi	>=0.135.1	ASGI web framework hosting the Inngest worker
uvicorn	>=0.41.0	ASGI server for FastAPI
streamlit	>=1.55.0	Interactive web UI for upload and querying
inngest	>=0.5.18	Durable function orchestration and event bus
qdrant-client	>=1.17.1	Python SDK for the Qdrant vector database
sentence-transformers	>=5.3.0	Local embedding model (all-MiniLM-L6-v2)
llama-index-core	>=0.14.17	Document loading and node parsing framework
llama-index-readers-f ile	>=0.6.0	PDFReader plugin for LlamaIndex
openai	>=2.28.0	OpenAI-compatible client (used by Inngest AI adapter)
google-genai	>=1.67.0	Google Generative AI SDK
python-dotenv	>=1.2.2	Loads .env files into os.environ
pydantic	(transitive)	Data validation and serialisation for custom types

9. Known Issues & Future Improvements

Known Issues

• QdrantStorage uses self.dim but dim is not stored as an instance attribute in __init___— this will raise AttributeError on collection creation. Fix: add self.dim = dim in __init__.

• The Streamlit polling loop (wait_for_run_output) blocks the UI thread. Consider using st.empty() with async rerun or background threads.

• No deduplication: re-uploading the same PDF will upsert duplicate vectors (though deterministic UUIDs mitigate this for identical content).

• The Inngest AI adapter model name 'gemini-3.1-flash-lite-preview' should be verified against the current Google API — model names change frequently.

Future Improvements

• Add authentication to the Streamlit UI to restrict access.

• Support multiple collections / namespaces so documents can be grouped by project.

- Implement streaming LLM responses for a better UX during long answers.
- Add a document management page to list, preview, and delete ingested PDFs.
- Replace the polling approach with Inngest webhooks or WebSocket push for real-time results.
- Containerise all services (Compose file) for one-command local startup.
- Add evaluation harness (e.g., RAGAS) to measure retrieval and answer quality.